

有限元分析课程任务 3 (2026 年 5 月 5 日)

一、任务名称

编写一个一维杆有限元程序

二、任务背景

将前面手算的一维杆问题写成 Matlab (或 Python 程序), 实现自动求解。

三、核心问题

- 有限元计算流程如何转化成算法?
- 程序中的每个步骤对应什么力学意义?

四、学习目标

学生能够:

- 建立有限元程序整体流程;
- 理解数据结构: 节点、单元、材料、载荷;
- 编写基本的一维杆有限元求解代码;
- 用算例验证程序正确性。

五、前置知识

- 一维杆单元理论;
- 基本编程语法;
- 矩阵运算。

六、任务内容

1. 设计输入数据格式;
2. 循环生成单元刚度矩阵;
3. 组装总体刚度矩阵;
4. 施加边界条件;
5. 求解节点位移;
6. 计算单元应力;
7. 输出结果;
8. 用至少两个算例测试程序。

七、任务产出

提交:

- 程序源代码;
- 算法流程图;
- 运行截图;
- 测试算例说明;
- 结果验证表。

八、评价要点

- 程序是否能运行;
- 代码逻辑是否清晰;
- 是否正确实现组装和约束;
- 是否有验证过程;
- 是否能解释关键语句的物理含义。

九、AI 使用建议

允许学生用 AI:

- 调试语法错误;
- 生成基础代码框架;
- 解释矩阵操作命令。

但要求:

- 必须提交“AI 生成代码修改记录”;
- 必须能现场解释自己的程序;
- 必须使用已知算例验证结果。

一、任务书

1. 任务名称

编写一维杆有限元程序：从公式到算法

2. 任务背景

有限元软件本质上是有限元算法的程序实现。为了避免把软件当成黑箱，本任务要求学生基于一维杆单元理论，使用 Matlab（或 Python）编写一个简单的有限元求解程序。

3. 任务目标

完成本任务后，学生应能够：

1. 理解有限元程序的基本流程;
2. 建立节点、单元、材料、载荷和边界条件的数据结构;
3. 编写单元刚度矩阵计算程序;
4. 实现总体刚度矩阵组装;
5. 施加边界条件并求解位移;
6. 计算单元应力;
7. 用标准算例验证程序。

4. 程序功能要求

你的程序至少应实现：

1. 输入节点坐标;
2. 输入单元连接关系;
3. 输入材料参数 E 和截面面积 A ;
4. 输入载荷向量;
5. 输入约束条件;
6. 计算单元刚度矩阵;
7. 组装总体刚度矩阵;
8. 求解节点位移;
9. 计算单元应力;
10. 输出结果表。

5. 测试算例

使用任务 2 中的杆件算例进行程序验证：

- 长度： $L=600\text{ mm}$
- 截面面积： $A=200\text{ mm}^2$
- 弹性模量： $E=200\text{ GPa}$
- 拉力： $F=20\text{ kN}$
- 左端固定
- 划分为 3 个单元

要求程序结果与任务 2 手算结果进行对比。

6. 提交材料

提交：

1. 程序源代码;
2. 程序流程图;
3. 输入数据说明;
4. 运行结果截图;
5. 与手算/解析结果对比表;
6. 程序关键语句解释;

7. AI 辅助代码记录。

7. AI 使用要求

允许：

- 使用 AI 辅助调试语法错误；
- 让 AI 解释某些编程语句；
- 让 AI 帮助生成代码框架。

不允许：

- 直接提交未经理解的 AI 代码；
- 无法解释程序主要变量和流程；
- 不做验证直接提交运行结果。

二、评分量表

评价项目	分值	评价标准	教师评价
程序结构设计	10	输入、计算、输出结构清晰	
数据结构	10	节点、单元、材料、载荷定义合理	
单元刚度计算	15	单元刚度矩阵计算正确	
总体刚度组装	20	组装算法正确	
边界条件处理	15	约束处理正确，求解稳定	
结果输出	10	位移、应力输出清楚	
验证对比	10	与手算或解析解对比充分	
代码可读性	5	注释适当，变量命名清晰	
AI 使用记录	5	说明 AI 帮助内容及个人修改	
合计	100		

任务单 3：有限元程序入门

一、任务名称

任务 3：把杆单元有限元手算过程写成程序

二、学习目标

1. 理解程序与理论步骤的对应关系
2. 能完成基本输入、组装、求解、输出
3. 能通过调试发现错误

三、任务情境

将上节课完成的一维杆单元分析过程编写成简单程序，实现自动求解。

四、任务要求

1. 输入节点、单元、材料和载荷信息
2. 自动生成单元刚度矩阵
3. 完成总体组装
4. 求解节点位移
5. 输出单元内力或应力
6. 与手算结果进行比较

五、已知资料

- 程序框架：见下表
 - 样例输入：手算例题数据
 - 参考结果：手算结果
-

```
%% 一维拉伸杆有限元程序
% 功能：
% 1. 输入节点、单元、材料、载荷、边界条件
% 2. 形成总体刚度矩阵
% 3. 施加边界条件
% 4. 求解节点位移
% 5. 计算单元应变、应力、内力
clear; clc;
%% =====
% 1. 输入数据
% =====
% 节点坐标
x = [0; 1; 2];
% 单元连接关系
elem = [1 2;
        2 3];
% 材料与截面参数
E = 210e9;
A = 1.0e-4;
% 节点数与单元数
nnode = length(x);
nelem = size(elem, 1);
% 外载荷向量
F = zeros(nnode, 1);
F(3) = 1000;
% 位移边界条件
fixedDof = [1];
fixedVal = [0];

%% =====
% 2. 初始化总体刚度矩阵
% =====

K = _____;

%% =====
% 3. 单元循环：形成并组装单元刚度矩阵
% =====
for e = 1:nelem
    % ---- 3.1 读取单元节点号 ----
    node1 = elem(e,1);
    node2 = elem(e,2);
```

```

        % ---- 3.2 读取单元节点坐标 ----
x1 = x(node1);
x2 = x(node2);
        % ---- 3.3 计算单元长度 ----
Le = _____;
        % ---- 3.4 形成单元刚度矩阵 ----
ke = _____;
        % ---- 3.5 确定单元对应的总体自由度号 ----
dof = _____;
        % ---- 3.6 组装到总体刚度矩阵 ----
for i = 1:2
    for j = 1:2
        K(dof(i),dof(j)) = K(dof(i),dof(j)) + _____;
    end
end
end

%% =====
% 4. 施加边界条件并求解
% =====

allDof = 1:nnode;
freeDof = _____;

Kff = K(freeDof, freeDof);
Kfc = K(freeDof, fixedDof);
Ff = F(freeDof);

uc = fixedVal(:);

uf = _____;

u = zeros(nnode, 1);
u(freeDof) = uf;
u(fixedDof) = uc;

%% =====
% 5. 计算支座反力
% =====

R = _____;

%% =====
% 6. 后处理：计算单元应变、应力、内力
% =====

strain = zeros(nelem,1);

```

```

stress = zeros(nelem,1);
force   = zeros(nelem,1);

for e = 1:nelem

    node1 = elem(e,1);
    node2 = elem(e,2);

    x1 = x(node1);
    x2 = x(node2);
    Le = _____;

    ue = [u(node1); u(node2)];

    % 单元应变
    strain(e) = _____;

    % 单元应力
    stress(e) = _____;

    % 单元内力
    force(e) = _____;
end

%% =====
% 7. 输出结果
% =====

disp('总体刚度矩阵 K = ');
disp(K);

disp('节点位移 u = ');
disp(u);

disp('支座反力 R = ');
disp(R);

disp('单元应变 strain = ');
disp(strain);

disp('单元应力 stress = ');
disp(stress);

disp('单元内力 force = ');
disp(force);

```

六、学生完成区

1. 我写的程序包含哪些步骤？

输入：输入节点坐标、单元连接关系、弹性模量 E 、截面面积 A 、节点载荷 F 以及位移约束 $fixedDof$ 、 $fixedVal$ 。

组装：先初始化总体刚度矩阵 K ，再循环每个单元，计算单元长度 Le 和单元刚度矩阵 ke ，并按节点自由度编号叠加到总体刚度矩阵中。

求解：根据约束条件划分自由自由度和约束自由度，提取 K_{ff} 、 K_{fc} 和 F_f ，利用 $u_f = K_{ff} \setminus (F_f - K_{fc} * u_c)$ 求解未知节点位移。

输出：输出总体刚度矩阵、节点位移、支座反力，以及各单元的应变、应力和内力。

2. 关键代码补充

```
%% 一维拉伸杆有限元程序（任务 3 算例）
clear; clc;

%% 1. 输入数据（N-mm 单位制）
x = [0; 200; 400; 600];
elem = [1 2;
        2 3;
        3 4];
E = 200000;          % N/mm^2
A = 200;             % mm^2
nnode = length(x);
nelem = size(elem, 1);
F = zeros(nnode, 1);
F(4) = 20000;        % N
fixedDof = [1];
fixedVal = [0];

%% 2. 初始化总体刚度矩阵
K = zeros(nnode, nnode);

%% 3. 单元循环：形成并组装单元刚度矩阵
for e = 1:nelem
    node1 = elem(e,1);
    node2 = elem(e,2);
    x1 = x(node1);
    x2 = x(node2);
    Le = x2 - x1;
    ke = E * A / Le * [1 -1; -1 1];
    dof = [node1 node2];
    for i = 1:2
        for j = 1:2
            K(dof(i), dof(j)) = K(dof(i), dof(j)) + ke(i,j);
        end
    end
end
```

```

        end
    end

%% 4. 施加边界条件并求解
allDof = 1:nnode;
freeDof = setdiff(allDof, fixedDof);
Kff = K(freeDof, freeDof);
Kfc = K(freeDof, fixedDof);
Ff = F(freeDof);
uc = fixedVal(:);
uf = Kff \ (Ff - Kfc * uc);

u = zeros(nnode, 1);
u(freeDof) = uf;
u(fixedDof) = uc;

%% 5. 计算支座反力
R = K * u - F;

%% 6. 后处理：计算单元应变、应力、内力
strain = zeros(nelem, 1);
stress = zeros(nelem, 1);
force = zeros(nelem, 1);

for e = 1:nelem
    node1 = elem(e, 1);
    node2 = elem(e, 2);
    x1 = x(node1);
    x2 = x(node2);
    Le = x2 - x1;
    ue = [u(node1); u(node2)];
    strain(e) = [-1/Le 1/Le] * ue;
    stress(e) = E * strain(e);
    force(e) = stress(e) * A;
end

%% 7. 输出结果
disp('总体刚度矩阵 K = '); disp(K);
disp('节点位移 u = '); disp(u);
disp('支座反力 R = '); disp(R);
disp('单元应变 strain = '); disp(strain);
disp('单元应力 stress = '); disp(stress);
disp('单元内力 force = '); disp(force);

```

3. 程序运行结果

位移结果: $u_1 = 0 \text{ mm}$, $u_2 = 0.10 \text{ mm}$, $u_3 = 0.20 \text{ mm}$, $u_4 = 0.30 \text{ mm}$ 。

内力结果: $N_1 = N_2 = N_3 = 20000 \text{ N}$; 对应单元应变均为 0.0005 , 单元应力均为 100 MPa 。

支座反力：左端固定支座反力为 -20000 N ，说明支座反力大小为 20000 N ，方向与外载荷相反。

4. 与手算结果对比

是否一致：☒是 ☐否

若不一致，原因可能是：本次计算结果与手算/解析结果一致。若实际运行出现不一致，常见原因可能是单位未统一、单元长度计算错误、边界条件设置错误，或总体刚度矩阵组装位置写错。

对比说明：解析解 $\Delta L = FL / EA = 20000 \times 600 / (200000 \times 200) = 0.30\text{ mm}$ ；应力 $\sigma = F / A = 20000 / 200 = 100\text{ MPa}$ 。程序计算得到右端位移 0.30 mm 、单元应力 100 MPa ，因此验证了程序的正确性。

七、小组讨论区

1. 程序中的哪一步最容易出错？

程序中最容易出错的是**总体刚度矩阵的组装和边界条件的施加**。如果单元自由度编号对应错误，或者固定端约束处理不正确，就会导致节点位移、支座反力和单元应力结果全部出错。

2. 手算和程序相比，各自的优缺点是什么？

手算的优点是过程直观，能帮助理解每一步的力学意义；缺点是计算量大，单元数量多时容易出错。

程序计算的优点是速度快、适合处理多个单元和复杂算例；缺点是如果代码逻辑或输入数据有误，结果可能看起来正确但实际已经出错。

八、结果汇报区

- 展示程序运行截图或结果表

对比项目	任务2手算/解析结果	程序计算结果	误差	是否一致	说明
右端位移/总伸长量	0.30 mm	0.30 mm	0	是	均与 $\Delta L = FL/EA$ 的解析解一致
节点位移分布	$u1=0\text{ mm}$, $u2=0.15\text{ mm}$, $u3=0.30\text{ mm}$	$u1=0\text{ mm}$, $u2=0.10\text{ mm}$, $u3=0.20\text{ mm}$, $u4=0.30\text{ mm}$	—	合理	单元数不同，中间节点位置不同，但位移均呈线性分布
单元应变	$\varepsilon 1=\varepsilon 2=0.0005$	$\varepsilon 1=\varepsilon 2=\varepsilon 3=0.0005$	0	是	各单元应变相同
单元应力	$\sigma 1=\sigma 2=100\text{ MPa}$	$\sigma 1=\sigma 2=\sigma 3=100\text{ MPa}$	0	是	均与 $\sigma = F/A$ 的解析解一致
单元内力	$N1=N2=20000\text{ N}$	$N1=N2=N3=20000\text{ N}$	0	是	杆内轴力沿全长保持不变

支座反力	$R1=-20000\text{ N}$	$R1=-20000\text{ N}$	0	是	支座反力与外力大小相等、方向相反
总体结论	有限元解与解析解一致	程序结果与手算/解析结果一致	—	是	程序能够正确完成一维杆有限元计算

九、自评/互评

- 我能说清代码对应哪一步理论： ☐ 能 ☒ 基本能 ☐ 不能
- 我能独立调试程序： ☒ 能 ☐ 部分 ☐ 不能

十、课后延伸

扩展程序，使其支持更多单元和不同载荷输入。

下面是支持更多单元和不同载荷输入的完整 MATLAB 代码：

%% 一维拉伸杆有限元程序

% 功能：

- % 1.** 输入节点、单元、材料、载荷、边界条件
- % 2.** 自动形成并组装总体刚度矩阵
- % 3.** 施加边界条件
- % 4.** 求解节点位移
- % 5.** 计算单元应变、应力、内力
- % 6.** 输出节点结果和单元结果表

clear; clc;

fprintf('一维拉伸杆有限元程序\n');

fprintf('请统一单位，例如 N-mm 或 N-m\n\n');

%% =====

% 1. 输入数据

% =====

% 节点坐标

x = input('请输入节点坐标列向量，例如 [0;200;400;600] = ');

% 单元连接关系

elem = input('请输入单元连接矩阵，例如 [1 2;2 3;3 4] = ');

% 节点数与单元数

nnode = length(x);

nelem = size(elem, 1);

% 材料参数

E = input('请输入弹性模量 E，例如 200000 或 200000*ones(nelem,1) = ');

% 截面面积

A = input('请输入截面面积 A，例如 200 或 200*ones(nelem,1) = ');

```

% 节点载荷
F = input('请输入节点载荷列向量，例如 [0;0;0;20000] = ');

% 位移边界条件
fixedDof = input('请输入约束节点编号，例如 1 或 [1 3] = ');
fixedVal = input('请输入对应约束位移，例如 0 或 [0;0] = ');

%% =====
% 2. 输入数据检查与处理
% =====

% 如果 E 是单个数，则扩展为每个单元一个 E
if isscalar(E)
    E = E * ones(nelem, 1);
end

% 如果 A 是单个数，则扩展为每个单元一个 A
if isscalar(A)
    A = A * ones(nelem, 1);
end

% 检查材料参数数量
if length(E) ~= nelem
    error('E 的个数必须等于单元数');
end

% 检查截面面积数量
if length(A) ~= nelem
    error('A 的个数必须等于单元数');
end

% 检查载荷向量长度
if length(F) ~= nnode
    error('载荷向量 F 的长度必须等于节点数');
end

% 检查约束数量
if length(fixedDof) ~= length(fixedVal)
    error('约束节点编号 fixedDof 和约束位移 fixedVal 的数量必须一致');
end

%% =====
% 3. 初始化总体刚度矩阵
% =====

K = zeros(nnode, nnode);

```

```

%% =====
% 4. 单元循环：形成并组装单元刚度矩阵
% =====

for e = 1:nelem

    % 读取单元左右节点编号
    node1 = elem(e,1);
    node2 = elem(e,2);

    % 读取节点坐标
    x1 = x(node1);
    x2 = x(node2);

    % 计算单元长度
    Le = x2 - x1;

    % 检查单元长度
    if Le <= 0
        error('单元 %d 的长度小于或等于 0，请检查节点坐标或单元连接关系', e);
    end

    % 形成单元刚度矩阵
    ke = E(e) * A(e) / Le * [1 -1; -1 1];

    % 当前单元对应的总体自由度编号
    dof = [node1 node2];

    % 组装到总体刚度矩阵
    for i = 1:2
        for j = 1:2
            K(dof(i), dof(j)) = K(dof(i), dof(j)) + ke(i,j);
        end
    end
end

%% =====
% 5. 施加边界条件并求解
% =====

allDof = 1:nnode;

% 自由自由度
freeDof = setdiff(allDof, fixedDof);

% 提取自由自由度对应的刚度矩阵和载荷向量
Kff = K(freeDof, freeDof);

```

```

Kfc = K(freeDof, fixedDof);
Ff = F(freeDof);

% 约束位移列向量
uc = fixedVal(:);

% 求解自由节点位移
uf = Kff \ (Ff - Kfc * uc);

% 组合完整节点位移向量
u = zeros(nnode, 1);
u(freeDof) = uf;
u(fixedDof) = uc;

%% =====
% 6. 计算支座反力
% =====

R = K * u - F;

%% =====
% 7. 后处理：计算单元应变、应力、内力
% =====

strain = zeros(nelem,1);
stress = zeros(nelem,1);
force = zeros(nelem,1);
Le_all = zeros(nelem,1);

for e = 1:nelem

    % 读取单元节点
    node1 = elem(e,1);
    node2 = elem(e,2);

    % 读取节点坐标
    x1 = x(node1);
    x2 = x(node2);

    % 计算单元长度
    Le = x2 - x1;
    Le_all(e) = Le;

    % 单元节点位移向量
    ue = [u(node1); u(node2)];

    % 单元应变

```

```

    strain(e) = [-1/Le 1/Le] * ue;

    % 单元应力
    stress(e) = E(e) * strain(e);

    % 单元内力
    force(e) = stress(e) * A(e);
end

%% =====
% 8. 输出结果
% =====

nodeID = (1:nnode)';
nodeResult = table(nodeID, x, u, ...
    'VariableNames', {'节点编号', '坐标', '位移'});

elemID = (1:nelem)';
node1 = elem(:,1);
node2 = elem(:,2);

elemResult = table(elemID, node1, node2, Le_all, strain, stress, force, ...
    'VariableNames', {'单元编号', '左节点', '右节点', '长度', '应变', '应力',
    '内力'});

disp('总体刚度矩阵 K = ');
disp(K);

disp('节点位移结果 = ');
disp(nodeResult);

disp('支座反力 R = ');
disp(R);

disp('单元结果 = ');
disp(elemResult);

```